📄 Technical Research Report

Comparative Design, Implementation, and Empirical Evaluation of 6 Machine Learning Classifiers for User Movie Satisfaction Prediction on MovieLens 32M

> **Module:** IT2011 - Artificial Intelligence and Machine Learning

> **Faculty:** Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT)

> **Benchmark Dataset:** MovieLens 32M (`ml-32m`)

> **Validation Protocol:** 5-Fold Stratified Cross-Validation with Bessel-Corrected Unbiased Sample Variance ($s^2 < 2.5 \times 10^{-5}$)

> **Consolidated Synthesis:** [`notebooks/group_model_comparison.ipynb`](notebooks/group_model_comparison.ipynb)

👥 Research Team & Individual Model Allocations

Member	Research Role	Classifier Paradigm	Deliverable Notebook
Divisekara A W D M D M B	**Project Architect & Lead**	**Member 2:** Decision Tree Classifier (Axis-Aligned Rule-Based Model)	[`Divisekara_Decision_Tree.ipynb`](notebooks/Divisekara_Decision_Tree.ipynb)
Kaweesha P M G S	Core Contributor	**Member 1:** Logistic Regression (Probabilistic Linear Classifier)	[`Kaweesha_Logistic_Regression.ipynb`](notebooks/Kaweesha_Logistic_Regression.ipynb)
Deshabhi R. G. S	Core Contributor	**Member 3:** Random Forest (Bagged Decision Tree Ensemble)	[`Deshabhi_Random_Forest.ipynb`](notebooks/Deshabhi_Random_Forest.ipynb)
Amadini G. G. A	Core Contributor	**Member 4:** Support Vector Machine (Linear vs RBF Maximum Margin)	[`Amadini_Support_Vector_Machine.ipynb`](notebooks/Amadini_Support_Vector_Machine.ipynb)
Hewage B H A S	Core Contributor	**Member 5:** K-Nearest Neighbors (Instance-Based Lazy Learner)	[`Hewage_K_Nearest_Neighbors.ipynb`](notebooks/Hewage_K_Nearest_Neighbors.ipynb)

Perera D.A.A.N.S	Core Contributor	**Member 6:** Multi-Layer Perceptron (Deep Neural Network)	[`Perera_Neural_Network_MLP.ipynb`](notebooks/Perera_Neural_Network_MLP.ipynb)

1. Introduction and Problem Statement

1.1 Problem Domain & Real-World Context

In the modern digital streaming economy (exemplified by Netflix, Disney+, Amazon Prime Video, and YouTube), user retention, subscription lifetime value, and engagement depend critically on algorithmic discovery. Modern streaming catalogs span tens of thousands of media titles, creating severe **information overload** for consumers. When users spend excessive time browsing without finding compelling content, churn rates surge.

To power recommendation feeds, streaming platforms must evaluate candidate items and predict which items have a high probability of user satisfaction. Formulating this challenge as a **Binary Classification Problem**:

$$ y = \begin{cases} 1 & \text{if } \text{rating} \ge 4.0 \text{ (Highly Satisfied / Recommended)} \\ 0 & \text{if } \text{rating} < 4.0 \text{ (Neutral / Disliked)} \end{cases} $$

allows the platform to rank and filter candidates before presenting the final carousel.

1.2 Multi-Paradigm Comparative Objective

Rather than testing an arbitrary mix of incompatible tasks, our research aligned all six team members on the exact same benchmark classification challenge on the MovieLens 32M dataset. This creates an apples-to-apples scientific comparison across six foundational AI/ML paradigms:

- **1.** **Linear Probabilistic Modeling:** Logistic Regression (Kaweesha)
- **2.** **Rule-Based Tree Partitioning:** Decision Tree (Divisekara)
- **3.** **Ensemble Bagging:** Random Forest (Deshabhi)
- **4.** **Geometric Maximum Margin:** Support Vector Machine (Amadini)

5. **Instance-Based Memory Learning:** K-Nearest Neighbors (Hewage)

6. **Hierarchical Non-Linear Representation:** Multi-Layer Perceptron Neural Network (Perera)

2. Dataset Description

2.1 Dataset Source & Provenance

Our group was assigned the **MovieLens 32M (`ml-32m`)** benchmark dataset generated by the **GroupLens Research Project** at the University of Minnesota (Harper & Konstan, 2015). The dataset captures authentic user interactions recorded between January 9, 1995, and October 12, 2023.

2.2 Schema & Structural Characteristics

The raw dataset comprises four relational files:

- **`ratings.csv` (837 MB, 32,000,204 rows):** 5-star ratings on a half-star scale (0.5 to 5.0). Attributes: `userId`, `movieId`, `rating`, `timestamp`.
- **`movies.csv` (4.1 MB, 87,585 rows):** Attributes: `movieId`, `title` (including release year), and `genres` (18 discrete pipe-separated categories).
- **`tags.csv` (70 MB, 2,000,072 rows):** Free-text user-generated metadata keywords capturing thematic and stylistic aspects.
- **`links.csv` (1.9 MB, 87,585 rows):** Foreign keys to IMDb and TMDb.

2.3 Curated High-Density Benchmark Formulation

To ensure fast, deterministic cross-validation without memory out-of-bounds errors on complex non-linear kernels, we extracted a high-density interaction core:

- 65,000 explicit ratings from 500 active users and 1,200 popular movies.
- Empirical class distribution: 57.2% negative/neutral ($y = 0$) and 42.8% positive satisfaction ($y = 1$).
- Median of ~130 ratings per user and ~50 ratings per movie, ensuring statistically rich feature representations.

3. Preprocessing & Exploratory Data Analysis (EDA)

3.1 Strict Preprocessing & Anti-Leakage Featurization

In strict adherence to Machine Learning best practices:

- **1.** **Featurization Ordering:** For all models, the dataset was partitioned into training and validation splits **prior** to fitting preprocessing transformers. Feature standardizers (`StandardScaler`) were fit exclusively on training folds.
- **2.** **Missing Value Imputation:** Missing release years in titles were parsed via regular expressions (`\((\d{4})\)`); remaining nulls were imputed with the catalog median year (1998).
- **3.** **Engineered Predictors:** Each interaction vector combined:
- User behavioral history: `user_mean_rating`, `user_rating_count`, `user_rating_std`.
- Movie reputation statistics: `movie_mean_rating`, `movie_rating_count`, `movie_rating_std`, `release_year`.
- Thematic vectors: 18 binary multi-hot genre indicators (Action, Comedy, Drama, Sci-Fi, etc.).

3.2 Exploratory Data Analysis Insights

- **Rating Skewness:** Mean rating was 3.54 ± 1.04. Users predominantly rate whole stars (3.0, 4.0, 5.0).
- **Feature Correlation:** Bivariate kernel density estimation confirmed strong separability: `movie_mean_rating` ($r \approx 0.45$) and `user_mean_rating` ($r \approx 0.38$) provide powerful predictive signals for user satisfaction.

Visual Analysis	Insight Summary	Output Asset

Genre Distribution	Catalog is heavily weighted toward Drama, Comedy, and Action	[`01_genre_distribution.png`](results/eda_visualizations/01_genre_distribution.png)
Rating Spread	Bimodal behavior with satisfaction threshold at 4.0 stars	[`02_rating_distribution.png`](results/eda_visualizations/02_rating_distribution.png)
PCA Clusters	2D projection reveals distinct user affinity groupings	[`03_kmeans_pca_clusters.png`](results/eda_visualizations/03_kmeans_pca_clusters.png)

4. Model Design and Implementation

We implemented six distinct machine learning algorithms covering all fundamental classification paradigms:

Classifier Paradigm	Individual Model	Lead Specialist	Core Inductive Bias
1. Linear Probabilistic	Logistic Regression	Kaweesha P M G S	Hyperplane log-odds mapping with L2 shrinkage
2. Rule-Based Partitioning	Decision Tree Classifier	Divisekara A W D M D M B	Recursive orthogonal split optimization (Gini impurity)
3. Bagged Ensemble	Random Forest Classifier	Deshabhi R. G. S	Bootstrap aggregation + random feature subspace decorrelation
4. Maximum Margin	Support Vector Machine (SVM)	Amadini G. G. A	Convex quadratic optimization of separating margin
5. Instance-Based Memory	K-Nearest Neighbors (KNN)	Hewage B H A S	Distance-weighted local neighborhood voting
6. Deep Neural Representation	Multi-Layer Perceptron (MLP)	Perera D.A.A.N.S	Non-linear hierarchical feature mapping via backpropagation

4.1 Member 1 (Kaweesha P M G S): Logistic Regression

- **Objective:** Establish a calibrated linear probabilistic benchmark.
- **Formulation:** Maps linear combinations into probabilities via the sigmoid function:

  $$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$
- **Varieties Evaluated:**
- *Variety A:* Weak regularization ($C = 100.0$).
- *Variety B:* Tuned balanced L2 regularization ($C = 1.0$).
- *Variety C:* Strong regularization ($C = 0.01$).
- **Notebook:** [`Kaweesha_Logistic_Regression.ipynb`](notebooks/Kaweesha_Logistic_Regression.ipynb)

4.2 Member 2 (Divisekara A W D M D M B): Decision Tree Classifier

- **Objective:** Construct white-box rule-based trees through recursive Gini impurity minimization:

  $$G(m) = 1 - \sum_{k=0}^1 p_{m,k}^2$$
- **Varieties Evaluated:**
- *Variety A:* Shallow tree ($d = 4, \text{min_samples_split} = 20$).
- *Variety B:* Tuned pruned tree ($d = 8, \text{min_samples_split} = 10$).
- *Variety C:* Deep unpruned tree ($d = 16, \text{min_samples_split} = 2$).
- **Notebook:** [`Divisekara_Decision_Tree.ipynb`](notebooks/Divisekara_Decision_Tree.ipynb)

4.3 Member 3 (Deshabhi R. G. S): Random Forest Classifier

- **Objective:** Eliminate tree variance through Bootstrap Aggregation (Bagging) and random feature subspace splitting.
- **Varieties Evaluated:**
- *Variety A:* Shallow ensemble ($n = 50, d = 6, \text{min_samples_split} = 20$).
- *Variety B:* Tuned ensemble ($n = 100, d = 12, \text{min_samples_split} = 5$).
- *Variety C:* Deep high-capacity ensemble ($n = 200, d = 20, \text{min_samples_split} = 2$).
- **Notebook:** [`Deshabhi_Random_Forest.ipynb`](notebooks/Deshabhi_Random_Forest.ipynb)

4.4 Member 4 (Amadini G. G. A): Support Vector Machine (SVM)

- **Objective:** Construct maximum-margin separating hyper-planes in Euclidean and Hilbert spaces.
- **Varieties Evaluated:**
- *Variety A:* Linear SVM ($C = 1.0$).
- *Variety B:* Non-linear RBF Kernel SVM ($C = 1.0, \gamma = \text{'scale'}$).
- *Variety C:* Non-linear RBF Kernel SVM ($C = 5.0, \gamma = \text{'scale'}$).
- **Notebook:** [`Amadini_Support_Vector_Machine.ipynb`](notebooks/Amadini_Support_Vector_Machine.ipynb)

4.5 Member 5 (Hewage B H A S): K-Nearest Neighbors (KNN)

- **Objective:** Predict satisfaction using distance-weighted voting across local training neighborhoods:

  $$P(y = 1 \mid \mathbf{x}) = \frac{\sum_{i \in N_k(\mathbf{x})} w_i \cdot \mathbb{I}(y_i = 1)}{\sum_{i \in N_k(\mathbf{x})} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i) + \epsilon}$$
- **Varieties Evaluated:**
- *Variety A:* Small neighborhood ($k = 5$, uniform weights, Euclidean L_2).
- *Variety B:* Tuned moderate neighborhood ($k = 15$, distance-weighted, Euclidean L_2).
- *Variety C:* Large neighborhood ($k = 35$, distance-weighted, Manhattan L_1).
- **Notebook:** [`Hewage_K_Nearest_Neighbors.ipynb`](notebooks/Hewage_K_Nearest_Neighbors.ipynb)

4.6 Member 6 (Perera D.A.A.N.S): Neural Network (Multi-Layer Perceptron / MLP)

- **Objective:** Learn high-order non-linear representations using feed-forward backpropagation networks with ReLU activations.
- **Varieties Evaluated:**
- *Variety A:* Single-layer architecture $(64,), \alpha = 0.01$.
- *Variety B:* Two-layer tuned architecture $(128, 64), \alpha = 0.001$, Early Stopping.
- *Variety C:* Three-layer deep architecture $(128, 64, 32), \alpha = 0.0001$.
- **Notebook:** [`Perera_Neural_Network_MLP.ipynb`](notebooks/Perera_Neural_Network_MLP.ipynb)

5. Model Evaluation and Comparison

5.1 Validation Protocol & Variance Metric Formulation

All models were evaluated under identical **5-Fold Stratified Cross-Validation**.

To rigorously quantify model stability, we computed the **Unbiased Sample Variance (s^2)** across the $K=5$ validation folds with Bessel's correction ($N-1 = 4$ degrees of freedom):

$$s^2 = \frac{1}{K-1} \sum_{k=1}^K (\text{F1}_k - \overline{\text{F1}})^2$$

All reported error bars depict the **Sample Standard Deviation ($s = \sqrt{s^2}$)**.

5.2 Comprehensive 6-Model Master Performance Matrix

Specialist Member	Model Paradigm	Optimum Variety	Mean Accuracy	Mean F1-Score	Fold Variance (s^2)	Fold Std Dev (s)	Mean ROC-AUC	Training Time (s)	I L (
Perera D.A.A.N.S	Neural Network (MLP)	Var B (128×64, Stop)	**0.7525**	**0.7394**	1.62×10^{-5}	0.0040	**0.8225**	5.20 s	1
Deshabhi R. G. S	Random Forest	Var B ($n=100, d=12$)	0.7510	**0.7385**	1.48×10^{-5}	0.0038	0.8219	3.80 s	2
Amadini G. G. A	Support Vector (SVM)	Var B (RBF, $C=1.0$)	0.7410	**0.7321**	1.76×10^{-5}	0.0042	0.8040	14.50 s	8
Divisekara A W D M D M B	Decision Tree	Var B (Pruned $d=8$)	0.7360	**0.7182**	2.42×10^{-5}	0.0049	0.7765	0.35 s	* m
Hewage B H A S	K-Nearest Neighbors	Var B ($k=15$, Dist)	0.7345	**0.7162**	2.20×10^{-5}	0.0047	0.7914	**0.10 s**	1
Kaweesha P M G S	Logistic Regression	Var B (Tuned $C=1.0$)	0.7325	**0.7084**		0.0046	0.7850	0.45 s	* m

					2.15×10^{-5}				

5.3 Comparative Analysis & Operational Trade-offs

- **1.** **Predictive Champions:**
- **Deep MLP (Perera: F1=0.7394, ROC-AUC=0.8225)** and **Random Forest (Deshabhi: F1=0.7385, ROC-AUC=0.8219)** emerged as top performers. Their ability to model high-order non-linear combinations across user generosity and movie reputation yielded superior ranking discrimination.
- **2.** **Speed & Latency Champions:**
- **Logistic Regression (Kaweesha: 0.12 ms)** and **Decision Tree (Divisekara: 0.15 ms)** provide orders of magnitude faster inference than KNN (11.2 ms) and SVM (8.4 ms).
- **3.** **Variance Stability:**
- Across all 6 models, 5-fold cross-validation sample variances remained strictly below 2.5×10^{-5} (standard deviations < 0.005), confirming statistical stability across data partitions.

6. Ethical Considerations and Bias Mitigation

Machine Learning algorithms deployed in entertainment streaming systems exert substantial influence over cultural visibility, economic monetization for creators, and user cognitive exposure.

6.1 Algorithmic Bias & The Popularity Loop

- **The Rich-Get-Richer Phenomenon:** Classifiers trained predominantly on popularity metrics risk creating feedback loops that starve independent cinema.
- **Mitigation Strategy:** Adding genre parity constraints and ϵ-greedy exploratory injection (10% exploration slots) guarantees exposure for long-tail films.

6.2 Echo Chambers & Privacy

- The MovieLens 32M dataset protects user confidentiality through synthetic anonymized integer identifiers (`userId`), omitting personally identifiable demographic attributes (race, age, gender, geographic location), thereby preventing demographic profiling.

7. AI Tool Usage Declaration and Transparency

In accordance with academic standards regarding transparent AI-assisted research and development:

Tool Name & Version	Supported Project Aspect	Extent of Use	Verification & Ownership Method
Google Gemini 3.8 Flash	Code architecture scaffolding, LaTeX formula	Moderate	All generated Python scripts were compiled and unit-tested in a local Python 3.12 environment

	drafting, and EDA visualization ideas		(`.venv`). Output numbers and graphs were empirically verified against raw MovieLens data.
Google Colab CLI	Remote runtime session management and environment configuration	Minimal	Checked authentication scopes, validated execution logs, and confirmed local fallback execution.
Python Scikit-Learn Docs	API reference for GridSearchCV and cross-validation variance formulas	Moderate	Cross-referenced against textbook formulas (Bessel's correction sample variance).

The research team takes full intellectual ownership of all models, architectural designs, code implementations, and empirical benchmark findings presented in this repository.

8. Reflections and Lessons Learned

- **1.** **Trade-off between Interpretability and Predictive Power:** Simple models (Logistic Regression, Decision Trees) offer direct rule transparency and sub-millisecond scoring, whereas complex non-linear architectures (Deep MLP, Random Forest) gain ~3% in F1-score at the expense of white-box explainability.
- **2.** **Featurization Ordering:** A critical lesson was ensuring feature scaling transformers were strictly fit on training splits within cross-validation folds to eliminate data leakage.
- **3.** **Production Architecture Recommendation:** A two-tier architecture using Logistic Regression or Decision Trees for rapid candidate pre-filtering followed by Deep MLP or Random Forest for top-10 personalized ranking achieves the optimal commercial balance.

9. References

- **1.** Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4), 19:1–19:19.
- **2.** Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- **3.** Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297.
- **4.** Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, 1(1), 81–106.

5. Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

6. He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Tat-Seng, C. (2017). Neural Collaborative Filtering. *Proceedings of the 26th International Conference on World Wide Web (WWW)*, 173–182.

7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.